

LLM-Driven Adaptive Robotic Manipulation for Complex Task Planning and Execution

Ghadi Nehme

*Department of Mechanical Engineering
Massachusetts Institute of Technology
Cambridge, USA
ghadi@mit.edu*

Abstract—This project explores the integration of Large Language Models (LLMs) and Vision-Language Models (VLMs) into robotic perception and action pipelines for high-level task planning and execution in a simulated kitchen environment. By leveraging affordance-based action selection and multimodal perception, the proposed system addresses the challenges of dynamic and unstructured environments. Custom-designed scenarios—stacking, sorting, cleaning, and table setup—demonstrate the robot’s ability to execute complex tasks guided by natural language instructions. Key contributions include a robust grasping pipeline utilizing bounding box and antipodal strategies, and a modular task execution framework combining perception, planning, and skill implementation (FIND, PICK, PLACE, OPEN, and CLOSE). Evaluation results highlight the system’s success in simpler tasks, while revealing limitations in perception and hardware that affect more complex scenarios. Despite these challenges, the approach showcases the potential of LLMs for resolving ambiguities in human-like commands and generalizing across diverse tasks. The link to the video can be found here.

I. INTRODUCTION AND RELATED WORK

A. Objective

The integration of robots into human environments has long been a compelling goal in the field of robotics, particularly in tasks requiring a combination of perception, planning, and action execution. Kitchen environments present unique challenges, as they require robots to navigate dynamic scenes, interact with diverse objects, and execute complex sequences of tasks. This report explores a novel methodology that leverages the capabilities of Large Language Models (LLMs) and advanced vision-language pipelines to enable high-level planning and execution of tasks in a simulated kitchen setting.

By combining affordance-based action selection and multimodal perception with the interpretability of LLM-driven task planning, this work aims to address the limitations of traditional rule-based systems in ambiguous and unstructured scenarios. Using a simulated kitchen environment with custom-designed scenes and objects, we evaluate the system’s ability to execute tasks such as sorting, stacking, cleaning, and setting up a table.

This report details the system pipeline, including perception, planning, and robotic skill execution, and provides a comprehensive analysis of its performance in various scenarios. The results highlight the strengths and limitations of the proposed system, offering insights into its potential for deployment in real-world applications.

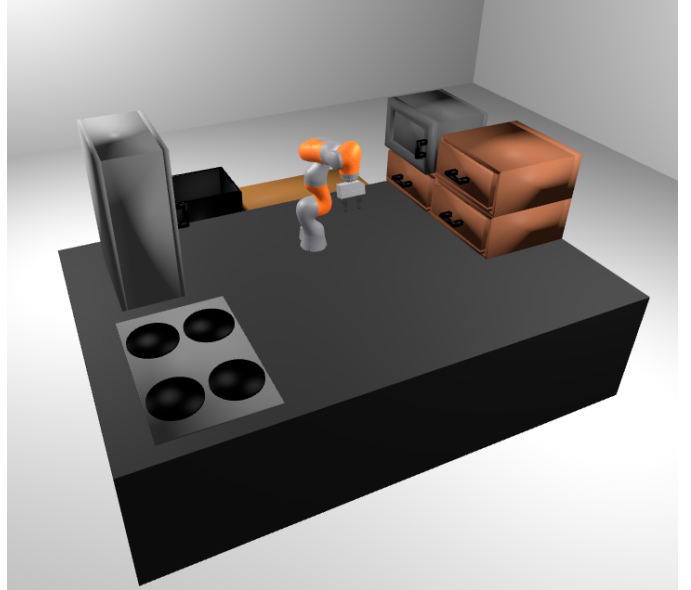


Fig. 1: A custom kitchen environment with drawers, a microwave, a trash bin, a fridge and a table

B. Related Work

Two key papers inform our approach. The first, “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances,” discusses how robots can enhance their understanding of human instructions by linking language with actionable affordances [1]. This work highlights the difficulty robots face in interpreting vague human language and the need for grounding commands in their capabilities and environment. The proposed model incorporates LLMs to manage ambiguity and variability in natural language, while also including an affordance-based action selection mechanism. This ensures that the robot selects feasible actions based on its physical capabilities and the environment, refining its actions through reinforcement learning (RL) and utilizing perception systems like cameras and depth sensors to analyze its surroundings.

The second relevant paper, “Lifelong Robot Learning with Human Assisted Language Planners,” explores the training of robots to perform complex tasks described in natural language using large-scale datasets [2]. This model leverages pre-trained language models and fine-tunes them on task-specific data,

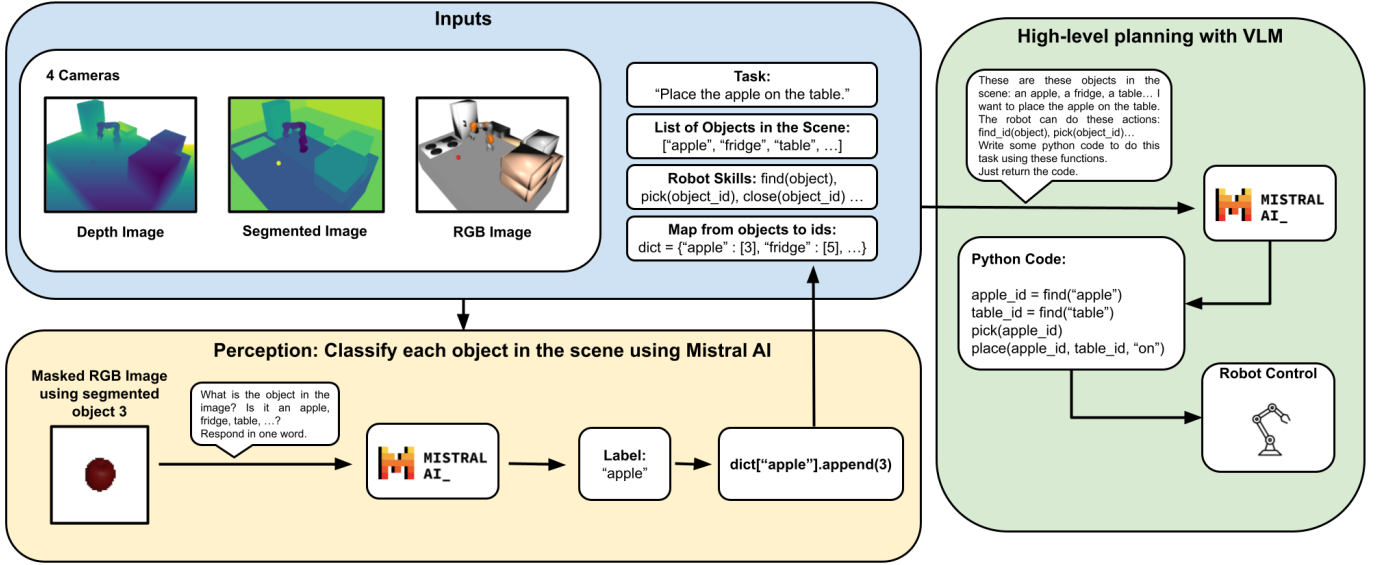


Fig. 2: Our pipeline for executing tasks with a robot: Inputs (camera images, task description, object list, and robot skills) are processed via a perception module using Mistral AI for object classification. High-level planning with a Vision-Language Model (VLM) generates Python code for execution, which the robot control module uses to perform tasks like placing an apple on a table.

allowing robots to generalize and respond effectively to human instructions without extensive real-time training. By using LLMs as language encoders, the model transforms human commands into embeddings that guide the robot’s action policies, enabling the robot to handle novel tasks.

II. APPROACH

This section describes the methodology employed to achieve high-level planning and robotic execution of kitchen tasks using a combination of a Large Language Model (LLM) and a robotic perception-action pipeline. We detail the constructed scenes, the multimodal capabilities of the Vision-Language Model (VLM) used for perception and planning, and the implementation of the robotic skills necessary for task execution: FIND, PICK, PLACE, OPEN, and CLOSE.

A. Scenes Description

The robotic tasks were developed within a simulated kitchen scene containing functional components such as drawers, a fridge, a microwave, a table, a trash bin, and four cameras positioned in the top corners of the room (Figure 1). Scenarios were generated by augmenting this base scene with objects randomly placed to test the robustness of the system. The following four scenes (Figure 3) were constructed:

- **Stacking Scene:** In this scene, the robot stacks cubes of different colors in a specific order.
- **Sorting Scene:** The robot sorts n bananas and m apples into drawers.
- **Cleaning Counter Scene:** Objects such as cups, bananas, apples, and cubes are placed on the counter, and the robot moves them to their designated locations (e.g., table, left drawer, right drawer, or trash bin).

- **Setup Table Scene:** The robot sets up a table with a fork, knife, plate, cup, and apple.

The objects in the scenes were custom-made using Solid-Works and exported as STL files. These were converted into OBJ files using MeshLab and finally into SDF format using the `manipulation.create_sdf_from_mesh` function. Functional components such as drawers, the fridge, and the microwave were made operable by adding prismatic and revolute joints.

B. Vision-Language Model (VLM): Perception and High-Level Planning

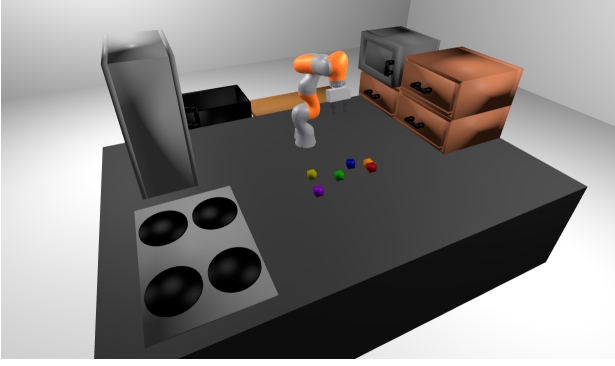
We utilize the Mistral AI multimodal model, Pixtral 12B [3], [4], which integrates a vision encoder and multimodal transformer for complex visual and text processing tasks. The VLM performs two critical functions: perception and high-level planning.

The perception pipeline begins by segmenting the RGB images captured from four cameras installed in the kitchen scene. Each object in the segmented image is classified using Mistral AI. Based on the classifications, a dictionary mapping object names to their IDs in the scene is created:

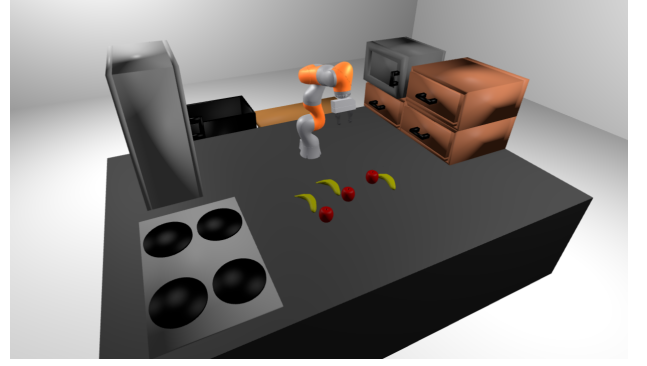
$$d = \{ \text{"apple"} : [3, 4], \text{"fridge"} : [5], \dots \}, \quad (1)$$

where the numerical values correspond to segmented object indices in the scene.

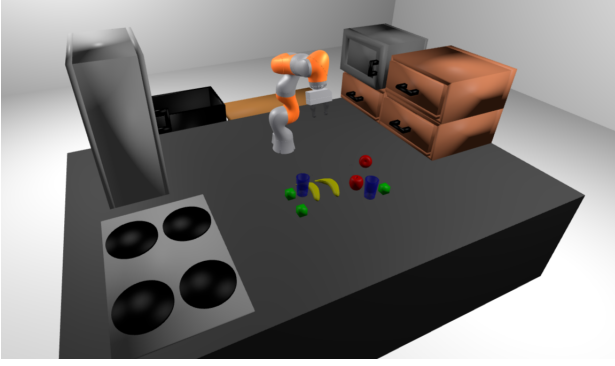
Given this mapping, the scene image, a list of object names, the robot’s skillset (e.g., `find`, `pick`, `place`, `open`, `close`), and a task description, the VLM generates Python code with high-level planning instructions for the robot. For example:



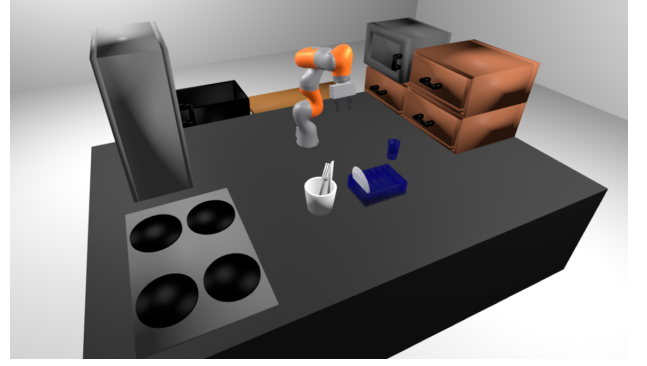
(a) Stacking Scene. Task example: “Stack the blocks in one pile in this order from bottom to top: Green, Red, Yellow, Blue, Orange, Purple”



(b) Sorting Scene. Task example: “Place the bananas in the bottom left drawer and the apples in the bottom right drawer”



(c) Cleaning Counter Scene. Task example: “Clear the counter by placing the cubes in the trash, the cups on the table, the bananas in the bottom left drawer and the apples in the bottom right drawer”



(d) Setup Table Scene. Task Example: “Setup the table and serve the apple that is in the fridge on the plate”

Fig. 3: Custom-designed kitchen task scenarios and an example task associated to each.

Example 1: “Place the apple on the plate”

```
apple_id = find("apple")
tray_id = find("plate")
pick(apple_id)
place(apple_id, "on", tray_id)
```

Example 2: “Take the apple from the fridge and place it on the table.”

```
plate_id = find("plate")
apple_id = find("apple")
fridge_id = find("fridge")
table_id = find("table")
pick(plate_id)
place(plate_id, "on", table_id)
open(fridge_id)
pick(apple_id)
place(apple_id, "on", plate_id)
close(fridge_id)
```

The generated code is executed sequentially, enabling the robot to perform the specified task. A diagram of the system pipeline is detailed in Figure 2.

C. *FIND(object_name)*

This function retrieves the object ID for a given name using the previously constructed dictionary. In depth algorithm architecture is detailed in Algorithm 1.

D. *PICK(object_id)*

The pick(object_id) function enables the robot to grasp objects using their segmented RGB-D images and point clouds (Figure 4). The process involves the following steps:

a) *Point Cloud Generation*: Segmented depth images from the four cameras of object_id are processed using the pinhole camera model to generate a point cloud of the segmented object:

$$X_c = \frac{(u - c_x)Z}{f_x}, \quad (2)$$

$$Y_c = \frac{(v - c_y)Z}{f_y}, \quad (3)$$

$$Z_c = Z \quad (4)$$

where (X_c, Y_c, Z_c) are points in the camera frame, f_x and f_y are camera focal lengths, c_x and c_y are the coordinates of the

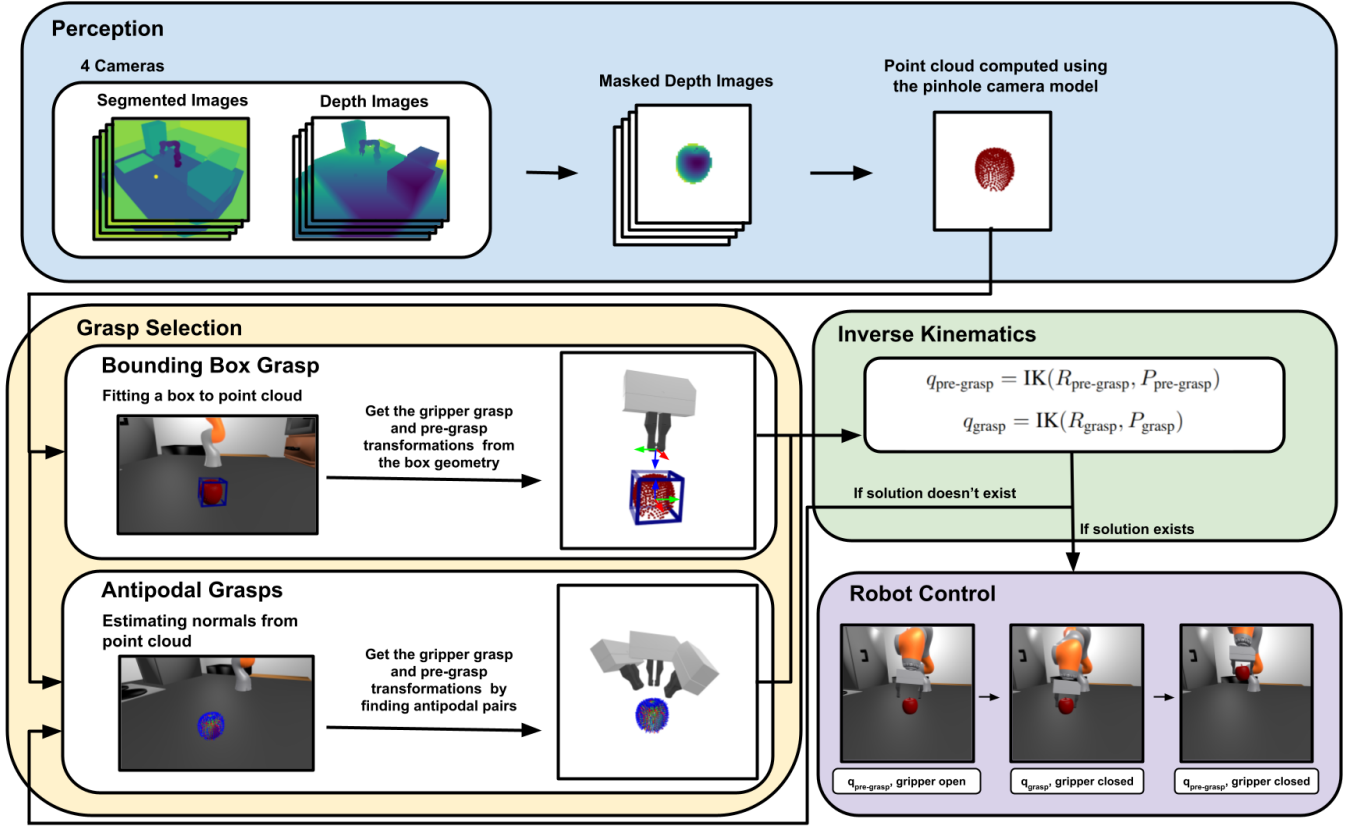


Fig. 4: PICK(object_id) planning pipeline: Perception generates point clouds from segmented images. Grasp selection uses Bounding Box or Antipodal methods, validated by inverse kinematics. Robot control executes the grasp through pre-grasp, grasp, and homing stages.

principal point, u and v are the pixel coordinates of the point in the 2D image plane, and Z is the depth value at (u, v) .

They are then transformed into the world frame:

$$\begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix} = \mathbf{R}_C \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} + \mathbf{t}_C, \quad (5)$$

where R_C and t_C are respectively the rotation matrix and translation of the camera with respect to the world.

b) *Grasp Planning*: Two grasping strategies are implemented to select a suitable grasp pose:

- 1) **Bounding Box Grasp**: A bounding box enclosing the point cloud is computed by minimizing its volume while ensuring all points are contained within the box (Figure 4). The optimization problem is formulated as:

$$\underset{R, \mathbf{t}}{\text{minimize}} \quad V(R, \mathbf{t}) = \prod_{i=1}^3 (b_i^{\max} - b_i^{\min}) \quad (6)$$

$$\text{subject to: } \mathbf{p}'_j = R\mathbf{p}_j + \mathbf{t}, \quad \forall j \quad (7)$$

$$b_i^{\min} \leq p'_{j,i} \leq b_i^{\max}, \quad \forall j, i = 1, 2, 3 \quad (8)$$

Here, $\mathbf{p}'_j$ represents the transformed points in the bounding box frame. Once the bounding box is computed, the gripper's orientation is determined by selecting:

- The axis with the highest dot product with the z -axis as the gripper's approach direction.
- The axis corresponding to the smallest side length as the gripper's opening axis, ensuring a stable grasp.

- 2) **Antipodal Grasp**: This strategy computes point normals from the point cloud by identifying the direction with the least variance at each point. (Figure 4) A valid grasp pair is selected by finding points with opposing normals. The gripper's frame is defined based on the normals and the direction of maximum variance.

- c) *Grasp and Pre-Grasp Positioning*: In both strategies, the gripper's pre-grasp and grasp positions are defined as:

$$\mathbf{p}_{\text{grasp}} = \mathbf{c} - 0.1\mathbf{y}_G, \quad \mathbf{p}_{\text{pre-grasp}} = \mathbf{c} - 0.2\mathbf{y}_G$$

where $\mathbf{y}_G$ is the direction of pointing of the gripper.

- d) *Feasibility Check and Execution*: The bounding box grasp is tested for kinematic feasibility by solving the inverse kinematics problem. If it is infeasible, the antipodal grasp is attempted. The inverse kinematics solutions provide joint configurations for:

$$q_{\text{pre-grasp}} = \text{IK}(R_{\text{pre-grasp}}, P_{\text{pre-grasp}})$$

$$q_{\text{grasp}} = \text{IK}(R_{\text{grasp}}, P_{\text{grasp}})$$

A trajectory is constructed using piecewise polynomials every 0.01 seconds, transitioning through:

$$q_0 \rightarrow q_{\text{pre-grasp}} \rightarrow q_{\text{grasp}}$$

After grasping, the gripper closes, and the robot returns to the pre-grasp position before moving to a homing position to avoid collisions. In depth algorithm architecture is detailed in Algorithm 2.

E. PLACE(object_id1, object_id2, on_or_in, loc)

The `PLACE` function is responsible for positioning `object1` relative to `object2` in the scene. This function involves calculating the desired position of the gripper (Figure 5), solving the inverse kinematics problem, and generating a trajectory using piecewise polynomials. The placement logic is divided into two main cases: placing `on` an object or table and placing `in` a container.

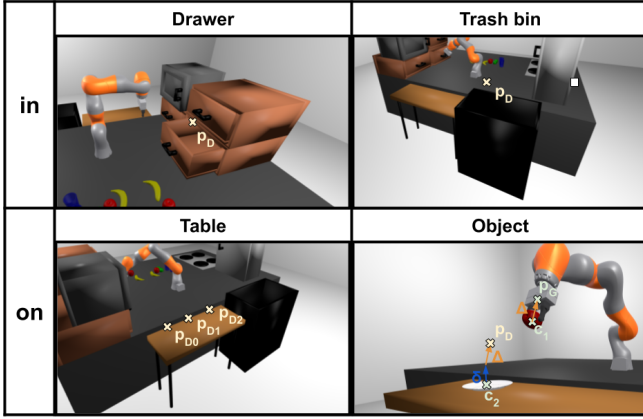


Fig. 5: The $P_{\text{gripper}}^{\text{desired}}$ of the `PLACE` Function for the different cases.

Placing Inside Containers: For the case of placing `object1` inside containers such as drawers or a trash bin, predefined gripper positions are utilized. These positions ensure that the gripper can accurately deposit the object within the confines of the container without collision.

Placing on a Table: When placing objects on a table, predefined gripper positions are used based on the `loc` parameter, which specifies the placement region (*left*, *center*, or *right*).

Placing on Another Object: Placing `object1` on `object2`, such as a plate or a cube, involves calculating the desired gripper position relative to the target object's center which is:

$$P_{\text{gripper}}^{\text{desired}} = C_2 + \Delta + \delta$$

where C_2 is the center of the target object, Δ is the offset from the current gripper position p_G to the center of `object1` C_1 , and δ is a predefined displacement vector. In depth algorithm architecture is detailed in Algorithm 3.

F. OPEN/CLOSE(object_id)

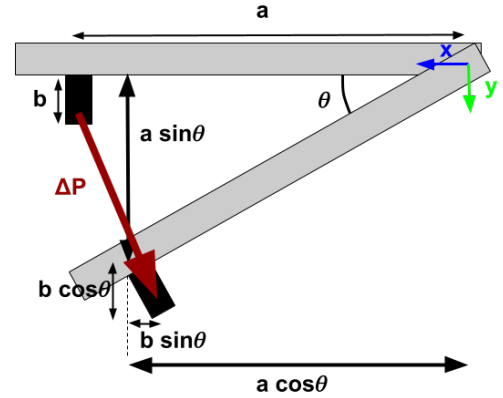
The `open` and `close` functions handle interactions with objects such as the fridge, microwave, and drawers. In depth algorithm architecture is detailed in Algorithm 4 and 5. These objects have either prismatic or revolute joints, and the functions involve solving the inverse kinematics problem with predefined orientation and translation constraints to ensure proper grasping and manipulation.

Handle Localization: Since the exact positions of handles are unknown, we use segmented depth maps to extract the corresponding point cloud. The pinhole camera model converts the point cloud into the world frame. The center of the point cloud is used to define the position constraint for the gripper.

Drawer Manipulation: For drawers, which operate using prismatic joints, the motion is straightforward. The gripper translates along the drawer's sliding axis while maintaining an orientation to ensure a secure grasp on the handle. The required translation vector is:

$$\Delta P = \lambda \hat{v}$$

where $\hat{v}$ is the normalized direction of the drawer's motion, and λ is the translation distance.



$$\Delta P = (a \cos(\theta) - b \sin(\theta), a \sin(\theta) + b \cos(\theta))$$

Fig. 6: Schematic of door `OPEN/CLOSE` viewed from above. The diagram illustrates the translation (ΔP) of the door handle relative to the base frame after the door rotates by an angle θ

Fridge and Microwave Manipulation: For the fridge and microwave, which operate using revolute joints, the gripper must follow a curved trajectory. This trajectory is defined based on the door's joint angle θ and the object's geometric dimensions. The translation of the fridge door handle relative to the base frame at a given door joint angle θ is given by:

$$\Delta P = [a \cos(\theta) - b \sin(\theta) - a, a \sin(\theta) + b \cos(\theta) - b, 0]$$

where a and b are some door and handle dimensions defined in figure 6. The gripper's orientation is continuously adjusted to maintain a grasp on the handle throughout the motion.

Close Function: The `close` function mirrors the opening trajectory in reverse. This ensures that the object returns to

its initial position without losing the grasp or causing any collisions.

G. Trajectory Execution

All robotic motions are planned as piecewise polynomial trajectories with waypoints sampled at 0.01-second intervals. For example, the trajectory for grasping is:

$$q(t) = \begin{cases} \text{interp}(q_0, q_{\text{pre-grasp}}, t), & 0 \leq t \leq T_1 \\ \text{interp}(q_{\text{pre-grasp}}, q_{\text{grasp}}, t), & T_1 < t \leq T_2 \end{cases}$$

where q_0 is the current joint configuration, and q_{grasp} is the desired configuration.

III. EVALUATION, RESULTS, AND DISCUSSION

The evaluation phase aimed to assess the performance of the proposed system in diverse scenarios, focusing on its robustness, adaptability, and failure modes. To ensure a comprehensive analysis, we compared two different grasping strategies, evaluated the adaptability of the Large Language Model (LLM) in handling complex tasks, and investigated the key failure modes observed during the experiments. Below, we present the results and discuss their implications.

A. Comparison of Bounding Box and Antipodal Grasp

To evaluate the performance of bounding box and antipodal grasp methods, we conducted 100 trials for each grasp type, using three objects: an apple, a banana, and a cup. The objects were placed randomly within the robot's workspace. The success rates for each grasp method are summarized in Table I.

Object	Bounding Box Grasp (%)	Antipodal Grasp (%)
Apple	95	86
Banana	93	81
Cup	90	85

TABLE I: Success rates of bounding box and antipodal grasp methods for different objects.

The results indicate that the bounding box method is generally more robust across all objects. However, its failure modes often involved infeasible grasps due to inverse kinematics (IK) issues, particularly when objects were close to the boundary of the workspace. This is likely due to the gripper's approach direction being nearly vertical (0, 0, -1), which is challenging for taller objects like the cup, explaining its slightly lower success rate.

In contrast, the antipodal grasp method exhibited failures primarily due to unstable grasps. These failures were attributed to inaccuracies in normal estimation from the point cloud, which could result in misaligned gripper orientations. Smaller objects like the banana were more susceptible to these issues because even slight misalignments could result in failed grasps.

B. Adaptability of the LLM in Complex Tasks

To evaluate the adaptability of the Large Language Model (LLM), we tested the system across four scenarios: stack, sort, clear counter, and setup table and their associated tasks (Figure 3). The layout of the environment remained constant, but object placements varied. Due to the time required for each task, we conducted 50 trials for the stack scenario, 30 for sort, and 10 each for clear counter and setup table. The results are presented in Table II. Some sample code generated by the LLM for the high-level planning is in Appendix V.

Scenario	Success Rate (%)	Number of Different Tasks	Trials Conducted
Stack	96	3	50
Sort	87	5	30
Clear Counter	80	6	10
Setup Table	60	7	10

TABLE II: Success rates of the LLM-driven pipeline across different scenarios.

The success rate decreased as task complexity increased. This is attributed to a higher probability of error accumulation in more complex scenarios. Notably, the generated code was inspected and found to be flawless in all trials. The observed failures were primarily due to external factors, as discussed below.

C. Failure Modes

Scenario 1: Stack The main failure mode was due to perception issues. When placing cubes on top of others, the gripper partially obscured the cubes, resulting in incomplete point clouds. This led to inaccurate center estimation and misaligned gripper positioning. These failures were rare, making the method robust overall.

Scenario 2: Sort Failures were primarily caused by hardware issues. For instance, the drawers failed to close correctly when objects, such as an apple, were placed inside. Adjustments to the physical parameters of the setup may be required to mitigate this issue.

Scenario 3: Clear Counter The main failure mode involved objects falling during transport between the pick-up and placement points. This occurred due to slightly unstable grasps, which worsened as the arm moved, leading to dropped objects.

Scenario 4: Setup Table Two significant failure modes were observed. First, apples occasionally rolled out of plates due to inaccuracies in perception when held by the gripper. Second, perception issues arose when picking apples from the fridge, as only one camera could view the object. This resulted in poor point cloud estimations and inaccurate grasp generation.

IV. CONCLUSION

This project presents a novel methodology that integrates Large Language Models (LLMs) and multimodal Vision-Language Models (VLMs) with robotic perception and action pipelines to enable high-level planning and execution

of tasks in a simulated kitchen environment. The system demonstrates promising results across four custom-designed scenarios—stacking, sorting, cleaning a counter, and setting up a table—highlighting its potential for addressing the challenges of unstructured and dynamic environments.

Key findings include the robustness of the bounding box grasping method, the adaptability of the LLM in handling diverse task complexities, and the limitations arising from perception and hardware-related issues. The evaluation revealed that the system achieves high success rates for simpler tasks (e.g., stacking) but struggles with more complex scenarios (e.g., table setup), where compounded errors and perception inaccuracies significantly affect performance.

Despite these limitations, the study underscores the potential of leveraging LLMs for task generalization and ambiguity resolution in human-like instructions. The approach’s modular design facilitates the integration of advanced perception, planning, and action execution methods, paving the way for future improvements.

Future work could focus on enhancing the perception pipeline to mitigate point cloud inaccuracies, incorporating real-time feedback to handle unforeseen errors, and extending the framework to real-world kitchen environments. Additionally, refining the grasping strategies and improving hardware reliability will be critical to increasing system robustness and scalability. This work serves as a foundation for further exploration into the integration of language models with robotic systems for real-world task automation.

REFERENCES

- [1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- [2] Meenal Parakh, Alisha Fong, Anthony Simeonov, Tao Chen, Abhishek Gupta, and Pulkit Agrawal. Lifelong robot learning with human assisted language planners. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 523–529. IEEE, 2024.
- [3] Mistral AI. Pixtral 12b: A vision-language model by mistral ai. <https://mistral.ai>, 2024. Accessed: October 30, 2024.
- [4] Mistral AI. Pixtral 12b: Vision-language model by mistral ai. <https://mistral.ai/pixtral-12b>, 2024. Multimodal AI model for text and image processing.

V. APPENDIX A: LLM PLANNING CODE FOR EACH SCENARIO

Here are some examples of code generated by the LLM for the scenarios and tasks presented in Figure 3.

Scenario 1: Stack, Task: “Stack the blocks in one pile in this order from bottom to top: Green, Red, Yellow, Blue, Orange, Purple”

```
green_cube_id = find("green_cube")
red_cube_id = find("red_cube")
yellow_cube_id = find("yellow_cube")
blue_cube_id = find("blue_cube")
orange_cube_id = find("orange_cube")
purple_cube_id = find("purple_cube")

pick(red_cube_id)
place(red_cube_id, green_cube_id, "on")

pick(yellow_cube_id)
place(yellow_cube_id, red_cube_id, "on")

pick(blue_cube_id)
place(blue_cube_id, yellow_cube_id, "on")

pick(orange_cube_id)
place(orange_cube_id, blue_cube_id, "on")

pick(purple_cube_id)
place(purple_cube_id, orange_cube_id, "on")
```

Scenario 2: Sort, Task: “Place the bananas in the bottom left drawer and the apples in the bottom right drawer”

```
left_drawer_id = find("bottom_left_drawer")
right_drawer_id = find("bottom_right_drawer")

open(right_drawer_id)
open(left_drawer_id)

for i in range(3):
    banana_id = find("banana")
    pick(banana_id)
    place(banana_id, left_drawer_id, "in")

for i in range(3):
    apple_id = find("apple_id")
    pick(apple_id)
    place(apple_id, right_drawer_id, "in")

close(right_drawer_id)
close(left_drawer_id)
```

Scenario 3: Clear Counter, Task: “Clear the counter by placing the cubes in the trash, the cups on the table, the bananas in the bottom left drawer and the apples in the bottom right drawer”

```
trash_id = find("trash")
```

```
for i in range(2):
    cube_id = find("cube")
    pick(cube_id)
    place(cube_id, trash_id, "in")

table_id = find("table_id")
for i in range(2):
    cup_id = find("cup")
    pick(cup_id)
    place(cup_id, table_id, "on")

left_drawer_id = find("bottom_left_drawer")
open(left_drawer_id)

for i in range(2):
    banana_id = find("banana")
    pick(banana_id)
    place(banana_id, left_drawer_id, "in")

close(left_drawer_id)

right_drawer_id = find("bottom_right_drawer")
open(right_drawer_id)

for i in range(2):
    apple_id = find("apple_id")
    pick(apple_id)
    place(apple_id, right_drawer_id, "in")

close(right_drawer_id)
```

Scenario 4: Setup Table, Task: “Setup the table and serve the apple that is in the fridge on the plate”

```
table_id = find("table")

plate_id = find("plate")
pick(plate_id)
place(plate_id, table_id, "on", "center")

cup_id = find("cup")
pick(cup_id)
place(cup_id, table_id, "on", "left")

fork_id = find("fork")
pick(fork_id)
place(fork_id, table_id, "on", "left")

knife_id = find("knife")
pick(knife_id)
place(knife_id, table_id, "on", "right")

fridge_id = find("fridge")
```

```

open(fridge_id)

apple_id = find("apple")
pick(apple_id)
place(apple_id, plate_id, "on")

close(fridge_id)

```

VI. APPENDIX B: PSEUDO CODE

Algorithm 1 Find Object Procedure

Input: object (name of the object to find)
Output: label_id (unique identifier for the object)

```

1: Function find_id(object):
2: Initialize:
3:   list_of_objects
4:   labels  $\leftarrow$  Predefined numerical IDs
5:   obj_dict  $\leftarrow$  Mapping objects to their labels
6: Step 1: Process Scene Data
7: for each label in labels do
8:   for each camera  $i \in \{0, 1, 2, 3\}$  do
9:     Retrieve segmented image from camera  $i$ :
10:    segmented  $\leftarrow$  Pixels in the segmented image
11:    if segmented region is large enough then
12:      Extract depth data for the segmented region:
13:      depth  $\leftarrow$  Cropped depth image
14:      Perform majority voting using a vision-language model:
15:      Shuffle list_of_objects and recognize object multiple times
16:      Select most common label as object name
17:      obj_dict[object_name].append(label)
18:    end if
19:  end for
20: end for
21: Step 2: Retrieve Object ID
22: Remove and return the next available label for the requested object from obj_dict:
23: label_id  $\leftarrow$  obj_dict[object].pop()
24: Return label_id

```

Algorithm 2 Object Picking Procedure

Input: object_id (Target Object ID)
Output: Grasp successfully executed

```

1: Step 1: Project depth image to point cloud and estimate normals.
2:  $pc \leftarrow$  project_depth_to_pC(4, object_id)
3: normals  $\leftarrow$  estimate_normals( $pc$ )
4: Step 2: Retrieve contexts and initialize joint configurations.
5:  $x_0 \leftarrow$  evaluated state vector of IIWA robot from the station output
6: Initialize  $q, q2 \leftarrow$  None
7: Step 3: Compute Grasp Candidates
8:  $q, q2 \leftarrow$  None
9:  $i \leftarrow 0$ 
10: while  $q = \text{None}$  or  $q2 = \text{None}$  do
11:   while valid grasp poses not found do
12:     Generate grasp poses using surface normals and object center.
13:     Use inverse kinematics to compute robot joint configurations.
14:   end while
15:    $i \leftarrow i + 1$ 
16: end while
17: Step 4: Execute Grasp
18: Move robot to pre-grasp pose with gripper open.
19: Move robot to grasp pose and close gripper.
20: Lift the object by returning to the pre-grasp pose.

```

Algorithm 3 Place Object Procedure

Input: object_id1 (object to place), object_id2 (target container), in_or_on (placement type)
Output: Object placed successfully

```

1: Function place(object_id1, object_id2, in_or_on):
2: if in_or_on == "in" then
3:   Select Target Position:
4:    $p_{WS} \leftarrow$  Target coordinates based on object_id2:
5:     Left Drawer:  $[-0.15, 0.55, 0.6]$ 
6:     Right Drawer:  $[0.35, 0.55, 0.6]$ 
7:     Trash:  $[-0.6, -0.5, 0.5]$ 
8:   Compute Gripper Pose:
9:    $R_{WG} \leftarrow$  RotationMatrix( $[G_x, G_y, G_z]^T$ )
10:   $X_G \leftarrow$  RigidTransform( $R_{WG}, p_{WS}$ )
11:  Move Robot:
12:   $x_0 \leftarrow$  Evaluated robot state from station output
13:   $q \leftarrow$  compute_joint_positions( $X_G$ )
14:  move_robot( $q$ , gripper_open = 0.0)
15:  Wait 1 second
16:  move_robot( $q$ , gripper_open = 0.1)
17: end if

```

Algorithm 4 Open Drawer Procedure

Input: object_id, system components (system, diagram_context, station)

Output: Drawer opened and closed successfully

```
1: Function open_drawer(object_id):
2: Step 1: Project depth image to point cloud and estimate normals.
3:  $pc \leftarrow \text{project\_depth\_to\_pC}(4, \text{object\_id})$ 
4:  $\text{normals} \leftarrow \text{estimate\_normals}(pc)$ 
5: Step 2: Retrieve contexts and initialize joint configurations.
6:  $x_0 \leftarrow$  evaluated state vector of IIWA robot from the station output
7: Initialize  $q, q2 \leftarrow \text{None}$ 
8: Step 3: Solve inverse kinematics to compute joint positions for opening the drawer.
9: while  $q = \text{None}$  or  $q2 = \text{None}$  do
10:   Compute gripper poses:  $X_G, X_{G2} \leftarrow \text{main}(\text{point\_cloud}, \text{normals}, \text{open\_drawer} = \text{True})$ 
11:   Solve for joint configurations:
12:    $q, q2 \leftarrow \text{compute\_joint\_positions}(X_G, X_{G2}, \dots)$ 
13: end while
14: Step 4: Move robot to execute drawer opening trajectory.
15:  $\text{move\_robot}(\text{station}, q2, \text{gripper\_open} = 0.07)$ 
16: Wait and adjust positions incrementally for smooth opening.
17: Step 5: Repeat process for closing the drawer.
18: Reproject point cloud, estimate normals, and compute joint configurations for closing.
19:  $\text{move\_robot}(\text{station}, \text{close\_drawer} = \text{True})$ 
```

Algorithm 5 Fridge Opening Procedure with Curved Trajectory

Input: object_id (Fridge Handle ID)

Output: Fridge Opened

```
1: Initialize System and Extract Data:
2: Extract the fridge handle's 3D point cloud, generate a corresponding point cloud object, and estimate surface normals.
3: Initialize Robot Context:
4: Retrieve the robot's current joint state and create references to the plant and station contexts.
5: Find Grasp Poses:
6: Repeat Until  $q \neq \text{None}$  and  $q2 \neq \text{None}$ :
7:   Compute potential grasp poses  $(X_G, X_{G2})$  using the main grasp generation routine.
8:   Calculate corresponding joint configurations  $q2$  and  $q$  using inverse kinematics.
9: Execute Initial Grasp:
10: Move Robot to pre-grasp pose  $q2$  with a slight gripper opening.
11: Move Robot to grasp pose  $q$  while maintaining gripper force.
12: Close Gripper and wait briefly.
13: Follow Curved Trajectory:
14: for  $i \leftarrow 0$  to 30 do
15:   Compute Sinusoidal Offsets:
16:    $dx \leftarrow 0.03 \cdot \sin\left(\frac{90-3i}{180}\pi\right) - 0.3 \cdot \cos\left(\frac{90-3i}{180}\pi\right)$ 
17:    $dy \leftarrow 0.03 \cdot \sin\left(\frac{90-3i}{180}\pi\right) + 0.3 \cdot \cos\left(\frac{90-3i}{180}\pi\right)$ 
18:   Update Gripper Pose:
19:   Set gripper pose translation:
    $X_{G3}.\text{set\_translation}(X_G.\text{translation}() + [dx, dy, 0])$ 
20:   Compute rotation angles  $a \leftarrow 50 - \frac{50i}{30}$ 
21:   Update orientation:
22:    $G_x \leftarrow [\cos(a\pi/180), \sin(a\pi/180), 0]$ 
23:    $G_y \leftarrow [\sin(a\pi/180), -\cos(a\pi/180), 0]$ 
24:    $G_z \leftarrow G_x \times G_y$ 
25:    $R_{WG} \leftarrow \text{RotationMatrix}([G_x, G_y, G_z]^T)$ 
26:    $X_{G3}.\text{set\_rotation}(R_{WG})$ 
27:   Move Robot to New Pose:
28:   Compute new joint positions  $q3$  using inverse kinematics.
29:   If  $q3 \neq \text{None}$ , execute movement to  $q3$ .
30: end for
```
